



RADIX

RED DE APRENDIZAJE Y DATOS EN
INFRAESTRUCTURAS EXTREMAS

CIUDAD GUAYANA, JULIO 2025

Índice

1. Introducción	4
1.1 Propósito	4
1.2 Alcance del producto	4
1.3 Definiciones, acrónimos y abreviaturas	4
1.4 Referencias	5
1.5 Visión general del documento	5
2. Fundamentos teóricos: funcionalidades de los sistemas distribuidos	5
2.1 Compartición de recursos	5
2.2 Transparencia	6
2.3 Apertura	6
2.4 Escalabilidad	6
2.5 Tolerancia a fallos	6
2.6 Concurrencia	7
2.7 Replicación y consistencia	7
2.8 Heterogeneidad	7
2.9 Seguridad	7
2.10 Correspondencia entre funcionalidades y requisitos	7
3. Descripción general	8
3.1 Perspectiva del producto	8
3.2 Funciones del producto (resumen)	8
3.3 Características de los usuarios	8
3.4 Restricciones generales	9
3.5 Suposiciones y dependencias	9
4. Requisitos funcionales	9
4.1 Módulo de autenticación y control de acceso	9
4.2 Módulo de cursos y lecciones	10
4.3 Módulo de quizzes y evaluación	10
4.4 Módulo de biblioteca multimedia	10
4.5 Módulo de foro	11
4.6 Módulo de gamificación	11
4.7 Módulo de sincronización DTN	11
4.8 Módulo de administración y observabilidad	12

5. Requisitos no funcionales	12
5.1 Operación offline y entorno	12
5.2 Rendimiento	12
5.3 Fiabilidad y disponibilidad	13
5.4 Seguridad	13
5.5 Usabilidad	13
5.6 Mantenibilidad	14
5.7 Portabilidad	14
5.8 Observabilidad	14
6. Restricciones de diseño e implementación	15
7. Trabajo futuro (fuera del alcance actual)	15

1. Introducción

1.1 Propósito

El propósito de este documento es especificar los requisitos funcionales y no funcionales del sistema RADIX, un sistema de gestión de aprendizaje (LMS, *Learning Management System*) que está diseñado para funcionar sin conexión a internet sobre un servidor de borde de bajo costo. El documento va dirigido al equipo de desarrollo y al personal docente que va a evaluar el proyecto.

1.2 Alcance del producto

RADIX es un LMS *offline-first* que simula un servidor de borde (Raspberry Pi) instalado en una comunidad remota de la Amazonía. El sistema le permite a un profesor administrar cursos, lecciones, quizzes y una biblioteca multimedia desde la red local, y los estudiantes pueden consumir ese contenido sin conexión externa, ganando experiencia (XP) y medallas. Todas las transacciones importantes se van registrando en una cola DTN (*Delay-Tolerant Networking*) para sincronizarlas con un servidor central cuando haya conectividad.

El sistema incluye: gestión de usuarios y roles, cursos con inscripción de estudiantes, lecciones en formato markdown con sintaxis de wiki-enlaces, quizzes con calificación, foro de discusión por curso, biblioteca de archivos multimedia, gamificación (XP/medallas), cola de sincronización DTN, y herramientas de administración (monitor del servidor y observabilidad de logs).

El sistema no incluye: el servidor central de sincronización (se simula el registro de transacciones, no la transmisión real), videoconferencia, ni acceso desde internet público.

1.3 Definiciones, acrónimos y abreviaturas

Término	Definición
LMS	<i>Learning Management System</i> , sistema de gestión de aprendizaje.
DTN	<i>Delay-Tolerant Networking</i> ; arquitectura de red tolerante a desconexiones prolongadas.
Servidor de borde	Equipo de cómputo local (p. ej. Raspberry Pi) que sirve la aplicación dentro de la red comunitaria.
RBAC	<i>Role-Based Access Control</i> , control de acceso basado en roles.
SPA	<i>Single-Page Application</i> , aplicación web de página única.
XP	Puntos de experiencia otorgados al estudiante por completar actividades.
Wiki-enlace	Sintaxis <code>[[id]]</code> dentro del contenido de una lección que embebe un archivo de la biblioteca, otra lección o un quiz.
FTS	<i>Full-Text Search</i> , búsqueda de texto completo (SQLite FTS5).

Término	Definición
Sincronización oportunista	Transferencia de datos que ocurre únicamente cuando la conectividad está disponible, sin garantía de momento exacto.
CRUD	Operaciones de creación, lectura, actualización y eliminación.

1.4 Referencias

- README del repositorio (README.md): descripción general, stack tecnológico y guía de instalación.
- IEEE Std 830-1998, *Recommended Practice for Software Requirements Specifications*.
- Tanenbaum, A. S. y Van Steen, M. (2017). *Distributed Systems: Principles and Paradigms* (3.ª ed.).
- Coulouris, G., Dollimore, J., Kindberg, T. y Blair, G. (2011). *Distributed Systems: Concepts and Design* (5.ª ed.). Addison-Wesley.

1.5 Visión general del documento

La sección 2 presenta la base teórica: las funcionalidades que debe tener un sistema distribuido y cómo se relacionan con este proyecto. La sección 3 describe el producto de forma general. La sección 4 lista los requisitos funcionales (RF) agrupados por módulo, y la sección 5 los requisitos no funcionales (RNF) organizados por atributo de calidad. Por último, la sección 6 lista las restricciones de diseño e implementación.

2. Fundamentos teóricos: funcionalidades de los sistemas distribuidos

Según Tanenbaum y Van Steen (2017), un **sistema distribuido** es una colección de computadores autónomos que se presenta ante sus usuarios como un sistema único y coherente. Sus componentes están en máquinas conectadas por una red y se comunican y coordinan únicamente pasando mensajes entre sí (Coulouris et al., 2011). RADIX entra en esta definición: hay un servidor de borde, varios clientes en la red local y un servidor central remoto que cooperan aunque la conectividad sea intermitente.

En la literatura se identifican varias funcionalidades y propiedades que un sistema distribuido debe cumplir. A continuación se describe cada una y cómo se aplica en este proyecto.

2.1 Compartición de recursos

El objetivo principal de un sistema distribuido es que varios usuarios puedan acceder a recursos comunes (hardware, datos, servicios) sin importar en qué máquina física se encuentren. El sistema tiene que controlar el acceso concurrente a esos recursos y mantenerlos disponibles a través de la red.

En RADIX: el servidor de borde comparte con todos los dispositivos de la comunidad un único repositorio de cursos, lecciones, archivos multimedia y foros. Los estudiantes acceden al mismo tiempo desde sus propios equipos por la red local.

2.2 Transparencia

El sistema debe ocultarle al usuario la separación física de sus componentes, para que todo se perciba como un solo sistema. Existen varios tipos: transparencia de **acceso** (las mismas operaciones sirven para recursos locales y remotos), de **ubicación** (no hace falta saber dónde está un recurso), de **concurrency** (varios usuarios trabajan sin interferirse), de **replicación** (el usuario no nota que existen copias), de **fallos** (el sistema sigue funcionando aunque falle una parte) y de **migración** (los recursos se pueden mover sin afectar su uso).

En RADIX: el estudiante consume lecciones, archivos y quizzes con una interfaz uniforme, sin saber si el dato está solo en el servidor de borde o ya se sincronizó con el central. Además, los wiki-enlaces `[[id]]` referencian los recursos por su identificador y no por su ubicación física.

2.3 Apertura

Un sistema distribuido es abierto cuando ofrece sus servicios mediante interfaces públicas y bien especificadas, basadas en estándares. Esto permite extenderlo o integrarlo con componentes de distintos orígenes.

En RADIX: toda la funcionalidad se expone por una API REST sobre HTTP con mensajes en JSON, que son estándares abiertos. Esto permite que cualquier cliente (la SPA actual u otro que se desarrolle después) consuma los mismos servicios.

2.4 Escalabilidad

El sistema debe poder crecer en cantidad de usuarios, de recursos o en dispersión geográfica sin que el rendimiento se degrade demasiado ni haya que cambiar la arquitectura. Tanenbaum distingue tres dimensiones: escalabilidad de **tamaño**, **geográfica** y **administrativa**.

En RADIX: el modelo de servidores de borde escala geográficamente replicando nodos: cada comunidad tiene su propio servidor autónomo, y para crecer se agregan más nodos en vez de agrandar uno central. Dentro de cada nodo, la API atiende a los usuarios concurrentes de la red local.

2.5 Tolerancia a fallos

Los sistemas distribuidos fallan de forma parcial: puede caerse un nodo o un enlace mientras el resto sigue funcionando. El sistema debe detectar los fallos, enmascararlos cuando se pueda, recuperarse y degradarse de forma controlada para mantener el servicio disponible.

En RADIX: la desconexión con el servidor central se considera el estado normal y no una falla: toda la operación sigue funcionando localmente y la cola DTN guarda las transacciones pendientes. Si el proceso se reinicia, las sesiones se recuperan desde disco y el apagado ordenado

termina las peticiones en curso. Si un cliente se desconecta de golpe, las escrituras en la base de datos no quedan a medias.

2.6 Concurrencia

Varios clientes acceden y modifican recursos compartidos al mismo tiempo. El sistema debe coordinar ese acceso para que los datos se mantengan consistentes, sin condiciones de carrera ni corrupción.

En RADIX: el backend atiende peticiones concurrentes de todos los usuarios de la red local. El acceso a la base de datos pasa por una capa que se probó bajo cargas de escritura concurrente sin corromperse, y las estructuras compartidas en memoria (sesiones, búfer de logs) están protegidas con exclusión mutua.

2.7 Replicación y consistencia

Los datos normalmente se replican en varios nodos para mejorar la disponibilidad y el rendimiento. Esto obliga a definir un modelo de consistencia, que va desde la consistencia fuerte (toda lectura ve la última escritura) hasta la **consistencia eventual**, donde las réplicas convergen con el tiempo y se aceptan divergencias temporales. Este último es el modelo natural cuando la red no es confiable.

En RADIX: el servidor de borde y el servidor central son réplicas que divergen mientras no hay conectividad. La cola DTN registra las transacciones locales y las propaga cuando se puede, garantizando que ambos lados converjan eventualmente sin bloquear la operación local.

2.8 Heterogeneidad

El sistema debe funcionar sobre redes, hardware, sistemas operativos y lenguajes distintos, usando protocolos y formatos de datos que no dependan de la plataforma.

En RADIX: clientes muy variados (celulares, tabletas, portátiles, con cualquier sistema operativo que tenga navegador) se comunican con un servidor ARM de bajo costo mediante HTTP/JSON. El backend compila sin dependencias nativas, así que se puede desplegar en x86 o ARM sin cambios.

2.9 Seguridad

Como los recursos viajan por una red y los usan varios actores, el sistema debe garantizar confidencialidad, integridad y disponibilidad: autenticar a los usuarios, autorizar cada operación y proteger las credenciales.

En RADIX: hay autenticación con credenciales y hash bcrypt, tokens de sesión en cada petición, y autorización por roles (RBAC) que el servidor verifica en cada endpoint.

2.10 Correspondencia entre funcionalidades y requisitos

Funcionalidad del sistema distribuido	Requisitos de RADIX que la materializan
Compartición de recursos	RF-08–RF-12, RF-19–RF-25

Funcionalidad del sistema distribuido	Requisitos de RADIX que la materializan
Transparencia	RF-12, RF-23, RNF-01
Apertura	RNF-20, RES-01
Escalabilidad	RNF-02, RNF-04, RNF-22
Tolerancia a fallos	RF-06, RF-29, RF-30, RNF-03, RNF-07–RNF-09
Concurrencia	RNF-07, RNF-08
Replicación y consistencia	RF-29–RF-31, RNF-03
Heterogeneidad	RNF-16, RNF-22, RNF-23
Seguridad	RF-01–RF-05, RNF-11–RNF-14

3. Descripción general

3.1 Perspectiva del producto

RADIX es un sistema autónomo compuesto por:

- **Backend:** API REST en Go 1.25 (framework Echo v5), con persistencia en libSQL/SQLite (sqlc + goose) e inyección de dependencias con uber/fx.
- **Frontend:** SPA en React 19 + TypeScript (Vite), servida en la red local y comunicada con el backend vía HTTP/JSON.
- **Base de datos:** servidor libSQL (`sqld`) local en contenedor Docker, o archivo SQLite local según configuración.

3.2 Funciones del producto (resumen)

El sistema ofrece gestión de contenido educativo (cursos, lecciones, quizzes y biblioteca multimedia), consumo offline con gamificación, un foro de discusión por curso, autenticación con tres roles, la cola de sincronización DTN y herramientas de administración y observabilidad del servidor.

3.3 Características de los usuarios

Rol	Descripción	Nivel técnico esperado
Administrador (Profesor)	Crea y gestiona todo el contenido; administra el servidor.	Medio; usa interfaces gráficas, no la terminal.
Estudiante	Consume cursos, presenta quizzes, participa en el foro, acumula XP.	Básico.
Invitado	Acceso de solo consulta sin credenciales.	Básico.

3.4 Restricciones generales

- Operación sin conexión a internet: toda funcionalidad principal debe ejecutarse íntegramente en la red local.
- Hardware objetivo de recursos limitados (Raspberry Pi): CPU ARM, memoria reducida, almacenamiento en tarjeta SD.
- Sin dependencia de servicios en la nube en tiempo de ejecución.

3.5 Suposiciones y dependencias

- Los dispositivos cliente (celulares, tabletas, portátiles) disponen de un navegador web moderno y acceso a la red local del servidor de borde.
- `ffprobe` (FFmpeg) está disponible en el servidor para la extracción de metadatos multimedia.
- La conectividad hacia el servidor central es intermitente e impredecible; el sistema nunca la asume disponible.

4. Requisitos funcionales

Cada requisito se identifica como **RF-XX**, con prioridad **Alta** (esencial), **Media** (importante) o **Baja** (deseable).

4.1 Módulo de autenticación y control de acceso

ID	Requisito	Prioridad
RF-01	El sistema debe permitir el inicio de sesión mediante correo electrónico y contraseña, verificando la contraseña contra un hash bcrypt almacenado.	Alta
RF-02	Ante credenciales inválidas, el sistema debe responder con un mensaje de error genérico único ("correo o contraseña inválidos"), sin revelar si el correo existe en el sistema.	Alta
RF-03	El sistema debe ofrecer un acceso como invitado sin credenciales, creando un usuario efímero con rol <code>guest</code> .	Media
RF-04	Tras un inicio de sesión exitoso, el sistema debe emitir un token de sesión (<i>Bearer token</i>) que el cliente enviará en cada petición autenticada.	Alta
RF-05	El sistema debe implementar control de acceso basado en roles (RBAC) con tres roles — administrador, estudiante e invitado — aplicado por middleware a todos los endpoints protegidos de la API.	Alta
RF-06	Las sesiones activas deben conservarse a través de reinicios del proceso servidor (persistencia del mapa de sesiones a disco al detenerse y recarga al iniciarse).	Media
RF-07	El sistema debe permitir cerrar la sesión, invalidando el token correspondiente.	Media

4.2 Módulo de cursos y lecciones

ID	Requisito	Prioridad
RF-08	El administrador debe poder crear, consultar, editar y eliminar cursos.	Alta
RF-09	El administrador debe poder crear, consultar, editar y eliminar lecciones dentro de un curso, con contenido en formato markdown.	Alta
RF-10	El administrador debe poder inscribir y desinscribir estudiantes en cada curso; los estudiantes solo deben acceder al contenido de los cursos en los que están inscritos.	Alta
RF-11	El editor de lecciones debe ofrecer resaltado de sintaxis para los wiki-enlaces <code>[[id]]</code> , vista previa al pasar el cursor y un panel lateral con los archivos enlazados.	Media
RF-12	El contenido de una lección debe poder embeber, mediante la sintaxis <code>[[id]]</code> , archivos de la biblioteca, otras lecciones y quizzes; el visor debe renderizarlos en línea según su tipo.	Alta
RF-13	El sistema debe registrar el avance del estudiante: marcar lecciones como completadas y otorgar los XP correspondientes.	Alta
RF-14	Los enlaces lección-a-lección deben materializarse en una tabla de vínculos (<code>lesson_links</code>) para permitir la navegación y consulta de referencias entre lecciones.	Media

4.3 Módulo de quizzes y evaluación

ID	Requisito	Prioridad
RF-15	El administrador debe poder crear, editar y eliminar quizzes con preguntas de opción múltiple, asociados a un curso e independientes de cualquier lección específica.	Alta
RF-16	El estudiante debe poder presentar un quiz y recibir su calificación inmediatamente al enviarlo.	Alta
RF-17	El sistema debe registrar las calificaciones de quizzes por curso y por estudiante, y otorgar los XP correspondientes.	Alta
RF-18	El administrador debe poder consultar las calificaciones de los estudiantes de un curso.	Media

4.4 Módulo de biblioteca multimedia

ID	Requisito	Prioridad
RF-19	El administrador debe poder subir archivos reales al servidor (video, audio, imagen, PDF, texto y documentos) y gestionarlos (consultar, editar metadatos, eliminar).	Alta

ID	Requisito	Prioridad
RF-20	Al subir un archivo multimedia, el sistema debe extraer automáticamente sus metadatos técnicos (duración y resolución) mediante ffmpeg.	Media
RF-21	El sistema debe ofrecer vista previa en línea de cada archivo según su tipo (reproductor de video/audio, visor de imagen/PDF/texto).	Media
RF-22	El sistema debe registrar qué usuario subió cada archivo y mostrar su nombre en la interfaz.	Baja
RF-23	El sistema debe informar, para cada archivo de la biblioteca, en qué lecciones está siendo utilizado (búsqueda en vivo de wiki-enlaces sobre el contenido de las lecciones).	Media

4.5 Módulo de foro

ID	Requisito	Prioridad
RF-24	Cada curso debe contar con un foro de discusión donde los usuarios inscritos puedan crear hilos y responder en ellos.	Media
RF-25	Los hilos y respuestas deben mostrar autor y fecha, y renderizar su contenido con formato.	Media

4.6 Módulo de gamificación

ID	Requisito	Prioridad
RF-26	El sistema debe acumular puntos de experiencia (XP) por estudiante al completar lecciones y quizzes.	Alta
RF-27	El sistema debe otorgar medallas (logros) en función del progreso del estudiante.	Media
RF-28	El estudiante debe disponer de un panel (<i>dashboard</i>) con su progreso: XP, medallas, cursos y avance por curso.	Media

4.7 Módulo de sincronización DTN

ID	Requisito	Prioridad
RF-29	El sistema debe registrar en una cola local todas las transacciones relevantes (avances, calificaciones) destinadas al servidor central.	Alta
RF-30	La cola debe operar bajo consistencia eventual: las transacciones permanecen encoladas indefinidamente hasta que exista conectividad, sin bloquear ninguna operación local.	Alta
RF-31	El administrador debe poder consultar el estado de la cola DTN y forzar manualmente un intento de sincronización.	Media

4.8 Módulo de administración y observabilidad

ID	Requisito	Prioridad
RF-32	El administrador debe disponer de un monitor del servidor con métricas en tiempo real: uso de disco, sesiones/usuarios activos y estado de la cola DTN.	Media
RF-33	El sistema debe ofrecer una página de logs con seguimiento en vivo (<i>live tail</i>) de los eventos del servidor.	Media
RF-34	El sistema debe conservar un historial durable de logs, filtrable por nivel de severidad y rango de fechas, y con búsqueda de texto completo (FTS5) sobre el mensaje.	Media
RF-35	El historial de logs debe depurarse automáticamente según una retención configurable (LOG_RETENTION_DAYS , 30 días por defecto).	Baja
RF-36	El sistema debe proveer un comando de siembra de datos de demostración (cmd/seed), separado del arranque del servidor e idempotente (no duplica datos si ya existen).	Baja

5. Requisitos no funcionales

Cada requisito se identifica como **RNF-XX**, agrupado por atributo de calidad.

5.1 Operación offline y entorno

ID	Requisito
RNF-01	Offline-first. Toda la funcionalidad principal del sistema debe ejecutarse sin conexión a internet, usando exclusivamente recursos de la red local. Ninguna operación de usuario puede depender de un servicio externo.
RNF-02	Hardware limitado. El sistema debe operar en un servidor de borde de bajos recursos (clase Raspberry Pi): binario backend único compilado, frontend estático, base de datos embebida o contenedor ligero.
RNF-03	Tolerancia a desconexión prolongada. La ausencia de conectividad hacia el servidor central por días o semanas no debe degradar ninguna función local; la sincronización es estrictamente oportunista (consistencia eventual).

5.2 Rendimiento

ID	Requisito
RNF-04	Autenticación sin costo de base de datos. La validación del token de sesión debe resolverse en memoria (mapa de sesiones), sin viaje a la base de datos en cada petición autenticada.

ID	Requisito
RNF-05	Logging sin impacto en latencia. La persistencia de logs debe ser asíncrona y por lotes (cada 3 s o 50 entradas); una base de datos lenta o caída nunca debe añadir latencia a una petición HTTP ni bloquear una llamada de log (descarte silencioso si el búfer se llena).
RNF-06	Respuesta interactiva en red local. Las operaciones habituales de la interfaz (navegar cursos, abrir lecciones, presentar quizzes) deben percibirse como inmediatas en la red local del servidor de borde.

5.3 Fiabilidad y disponibilidad

ID	Requisito
RNF-07	Integridad de datos ante concurrencia. El acceso a la base de datos debe resistir carga de escritura concurrente sin corrupción. (Decisión derivada: drivers de base de datos <i>pure-Go</i> vía HTTP sin réplica local, tras corrupciones reproducibles con el conector embebido previo.)
RNF-08	Resistencia a desconexión del cliente. La cancelación de una petición por parte del cliente (cierre de pestaña, navegación) no debe abortar transacciones de escritura en curso ni dejar la conexión de base de datos en estado inconsistente (contextos desacoplados de la cancelación).
RNF-09	Apagado ordenado. Ante SIGINT/SIGTERM, el servidor debe drenar las peticiones HTTP en curso, persistir las sesiones activas y cerrar la base de datos limpiamente.
RNF-10	Integridad referencial. La base de datos debe imponer restricciones de clave foránea y unicidad; las operaciones de la aplicación deben respetarlas (p. ej. la actualización de un archivo de biblioteca nunca sobrescribe la referencia al usuario que lo subió).

5.4 Seguridad

ID	Requisito
RNF-11	Las contraseñas deben almacenarse exclusivamente como hash bcrypt; nunca en texto plano ni con hash reversible.
RNF-12	Los mensajes de error de autenticación no deben permitir enumerar cuentas existentes (respuesta 401 idéntica para usuario inexistente y contraseña incorrecta).
RNF-13	Todo endpoint de la API, salvo los de inicio de sesión, debe exigir un token de sesión válido y verificar la autorización por rol antes de ejecutar la operación.
RNF-14	Los orígenes permitidos para CORS deben ser configurables por entorno.

5.5 Usabilidad

ID	Requisito
RNF-15	La interfaz debe estar en español, orientada a usuarios de nivel técnico básico (estudiantes de comunidades rurales).

ID	Requisito
RNF-16	La interfaz debe ser una SPA responsiva utilizable desde navegadores de dispositivos heterogéneos (celulares, tabletas, portátiles) conectados a la red local.
RNF-17	El invitado debe poder explorar el contenido con un solo clic, sin proceso de registro.

5.6 Mantenibilidad

ID	Requisito
RNF-18	Arquitectura desacoplada. El backend debe usar inyección de dependencias (uber/fx) con una única raíz de composición; los módulos dependen de interfaces mínimas propias, no de tipos concretos compartidos.
RNF-19	SQL tipado y migraciones versionadas. Todo acceso a datos debe generarse con sqlc a partir de consultas SQL declaradas, y todo cambio de esquema debe expresarse como migración goose versionada.
RNF-20	Respuestas HTTP uniformes. Todos los handlers deben producir respuestas con una forma consistente (éxito: payload directo; error: <code>{"error": "..."} </code>) a través de un paquete centralizado (<code>httpx</code>).
RNF-21	Logging estructurado unificado. Todo log de la aplicación debe emitirse por un único logger estructurado (zap) que alimenta simultáneamente consola, live-tail en memoria e historial durable.

5.7 Portabilidad

ID	Requisito
RNF-22	El backend debe compilar sin CGO (drivers de base de datos <i>pure-Go</i>), garantizando compilación cruzada trivial hacia la arquitectura ARM del servidor de borde.
RNF-23	El modo de persistencia (archivo SQLite local o servidor libSQL remoto) debe seleccionarse únicamente por variables de entorno, sin cambios de código.

5.8 Observabilidad

ID	Requisito
RNF-24	El esquema de logs debe ser agnóstico a la fuente: columnas fijas solo para marca de tiempo, nivel y mensaje; los datos estructurados de cada punto de llamada viajan en un campo JSON genérico.
RNF-25	Todo mensaje de log debe ser texto legible y significativo redactado por el desarrollador (nunca vacío), para que la búsqueda de texto completo sobre el historial sea útil.

6. Restricciones de diseño e implementación

ID	Restricción
RES-01	Backend en Go 1.25 con Echo v5; frontend en React 19 + TypeScript con Vite; estilos con Tailwind CSS.
RES-02	Persistencia en SQLite/libSQL; en desarrollo, contenedor <code>sqlc</code> local vía Docker Compose.
RES-03	Los identificadores de entidades son UUID generados por la aplicación (con la excepción documentada de <code>users.id</code> , provisto por el llamador).
RES-04	La relación lección ↔ archivo de biblioteca no se materializa en tabla: se resuelve en vivo buscando la sintaxis <code>[[id]]</code> en el contenido, eliminando la necesidad de mantenimiento de una tabla de unión.
RES-05	Las sesiones no se almacenan en la base de datos (decisión de rendimiento); se mantienen en memoria con snapshot a archivo JSON entre reinicios.
RES-06	La extracción de metadatos multimedia depende de la presencia de <code>ffprobe</code> en el servidor.

7. Trabajo futuro (fuera del alcance actual)

- Suite de pruebas automatizadas (unitarias y de integración), que todavía no existe.
- Expiración de sesiones (TTL): actualmente las sesiones no caducan.
- Implementación real del lado receptor de la sincronización DTN (el servidor central).
- Filtrado del historial de logs por campos estructurados (`fields` JSON); se evaluó pero se pospuso.